

NHISC

第六堂社課

Python基礎-2

題目一：程式碼分析

程式碼邏輯拆解：

1. 初始化：建立大小 200 的陣列 Q，填入 1~30。
2. 迴圈條件：`while tail > head + 1`（當佇列中剩餘超過 1 個元素時繼續）
3. 計數邏輯：
 - 取出 head 元素。
 - count 累加。
 - 關鍵判斷：若 `count == 3`，歸零並將元素放回 tail；否則丟棄

```
Q = [0] * 200
val = 0
count = 0
head = tail = 0
for i in range(1, 31):
    Q[tail] = i
    tail = tail + 1
```

題目一：程式碼分析

- 初始狀態：1, 2, 3, ..., 30
- 規則：每數到第 3 個保留，其餘丟棄。
- 第一輪執行結果：
 - 1 (count=1, 丟棄)
 - 2 (count=2, 丟棄)
 - 3 (count=3, 保留, 放回 tail)
 - ...
- 第一輪留下的數字：3 的倍數
- [3, 6, 9, 12, 15, 18, 21, 24, 27, 30]
 - 剩餘個數：10 個

```
while tail > head + 1:
    val = Q[head]
    head = head + 1
    count = count + 1
    if count == 3:
        count = 0
        Q[tail] = val
        tail = tail + 1
```

題目一：程式碼分析

- 第二輪篩選（接續上一輪，重新數數）：
 - 3（丟） 6（丟） 9（留）
 - 12（丟） 15（丟） 18（留）
 - 21（丟） 24（丟） 27（留）
 - 30（count=1，丟），注意：最後 30 報數為 1，目前的 count 停在 1
- 目前的佇列：9 18 27（且 count 目前累積為 1）
- 第三輪篩選：
 - 9 （count=2，丟）
 - 18 （count=3，留）
 - 27 （count=1，丟）
 - 最終結果：剩下 18

答案：B

```
while tail > head + 1:
    val = Q[head]
    head = head + 1
    count = count + 1
    if count == 3:
        count = 0
        Q[tail] = val
        tail = tail + 1
```

題目二：前置計算與觀念

- 題目資訊：
 - 陣列 $a[k] = 3k + 1$ （等差數列）。
 - 目標 $value = 100$
- 尋找目標索引值（Target Index）：
 - 解方程式： $3k + 1 = 100$
 - $3k = 99$
 - $k = 33$
 - 結論：我們要找的目標在索引 33 的位置

計算 $n1$ ：

索引 0 到 33 共有 $33 - 0 + 1 = 34$ 個數字

$n1 = 34$

	5	15	20	25	30	35	40	45	50
index	0	1	2	3	4	5	6	7	8

題目二：前置計算與觀念

程式碼行為：利用 low, high, mid 縮小範圍

次數 (n2)	low	high	mid (int)	a[mid] vs 100	結果
1	0	99	49	148 > 100	high = 48
2	0	48	24	73 < 100	low = 25
3	25	48	36	109 > 100	high = 35
4	25	35	30	91 < 100	low = 31
5	31	35	33	100 == 100	Found!

答案：D

題目三：程式碼分析與錯誤找出

致命錯誤：任何數乘以 0 都變 0

原本 0! 應該是 1，但這裡變成了 0

答案：B

題目四（陣列交換陷阱）

執行過程：

- 前半段（ $i=0$ 到 3）：頭尾互換，陣列逐漸「反轉」。
- 中間點（ $i=4$ ）：自己跟自己換（沒變）。
- 後半段（ $i=5$ 到 8）：又把剛剛換過的位置再換回來！

```
a = [1, 3, 5, 7, 9, 8, 6, 4, 2]
n = 9

for i in range(n):
    a[i], a[n-i-1] = a[n-i-1], a[i]
```

題目四（陣列交換陷阱）

- $i = 0, 1, 2, 3, 4$
- 輸出配對：
- $i=0$: 印 $a[0], a[8]$? 1, 2
- $i=1$: 印 $a[1], a[7]$? 3, 4
- $i=2$: 印 $a[2], a[6]$? 5, 6
- $i=3$: 印 $a[3], a[5]$? 7, 8
- $i=4$: 印 $a[4], a[4]$? 9, 9

```
a = [1, 3, 5, 7, 9, 8, 6, 4, 2]
n = 9
```

```
for i in range(n//2 + 1):
    print(a[i], a[n-i-1], end = ' ')
```

題目五（巢狀迴圈圖形）

- 目標圖形：倒三角形
- Line 0: ***** (6 星)
- Line 1: **** (4 星)
- Line 2: ** (2 星)
- 星星數量的數學公式：
- 觀察：
 - $i=0$? 6
 - $i=1$? 4
 - $i=2$? 2
- 公式：星星數 = $6 - 2i$

答案：(C)

請問下列程式碼執行後，螢幕印出的內容為何？

```
def check(n):  
    print("Run", end=" ")  
    return n >= 10  
  
val = 5  
if val > 10 and check(val):  
    print("A")  
else:  
    print("B")
```

(A) RunB

(B) RunA

(C) B

(D) A

```
def check(n):  
    print("Run", end=" ")  
    return n >= 10  
  
val = 5  
if val > 10 and check(val):  
    print("A")  
else:  
    print("B")
```

(A) RunB

(B) RunA

(C) B

(D) A

解析（短路求值 Short-circuit）：判斷式為 `val > 10 and check(val)`。

1. 先檢查 `val > 10` (`5 > 10`)，結果為 `False`。

2. 對於 `and` 來說，只要有一個是 `False`，結果就是 `False`。

3. Python（以及 C/C++）為了效率，不會執行後面的 `check(val)`。

4. 所以 "Run" 不會被印出來，直接跳到 `else` 印出 "B"。

答案：(C)

28. 給定右側函式 $F()$ ， $F()$ 執行完所回傳的 x 值為何？

- (A) $n(n+1) \sqrt{\log_2 n}$
- (B) $n^2(n+1)/2$
- (C) $n(n+1) [\log_2 n + 1]/2$
- (D) $n(n+1)/2$

```
def F(n):  
    x = 0  
    for i in range(1, n+1):  
        for j in range(i, n+1):  
            k = 1  
            while k <= n:  
                x = x + 1  
                k = k * 2  
    return x
```

28. 給定右側函式 $F()$ ， $F()$ 執行完所回傳的 x 值為何？

- (A) $n(n+1) \sqrt{\log_2 n}$
- (B) $n^2(n+1)/2$
- (C) $n(n+1) [\log_2 n + 1]/2$
- (D) $n(n+1)/2$

```
def F(n):  
    x = 0  
    for i in range(1, n+1):  
        for j in range(i, n+1):  
            k = 1  
            while k <= n:  
                x = x + 1  
                k = k * 2  
    return x
```

氣泡排序法 (Bubble Sort)

假設要從小排到大

- 核心概念：
 - 就像水底的氣泡往上浮一樣
 - 每一次回合，最大的數字會透過不斷的「兩兩交換」，浮到陣列的最右邊
- 相鄰比較，大的往後換

複雜度分析：

- 最差情況： $O(N^2)$ (資料是倒序的)
- 最佳情況： $O(N)$ (資料已經排好)

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n - 1):  
        for j in range(n - 1 - i):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
    return arr
```



[Watch video on YouTube](#)
Error 153
Video player configuration error



氣泡排序法 (Bubble Sort)

舉例：

對陣列 [3, 2, 5, 1] 進行由小到大氣泡排序，請問第一回合 (i=0) 結束後的陣列內容為何？

執行過程：

- 第一輪：

- 比較 2, 3

?

 交換

?

 [2, 3, 5, 1]
- 比較 3, 5

?

 不換

?

 [2, 3, 5, 1]
- 比較 5, 1

?

 交換

?

 [2, 3, 1, 5]

- 第二輪：

- 比較 2, 3

?

 不換

?

 [2, 3, 1, 5]
- 比較 3, 1

?

 交換

?

 [2, 1, 3, 5]
- 比較 3, 5

?

 不換

?

 [2, 1, 3, 5]

- 第三輪：

- 比較 2, 1

?

 交換

?

 [1, 2, 3, 5]
- 比較 2, 3

?

 不換

?

 [1, 2, 3, 5]
- 比較 3, 5

?

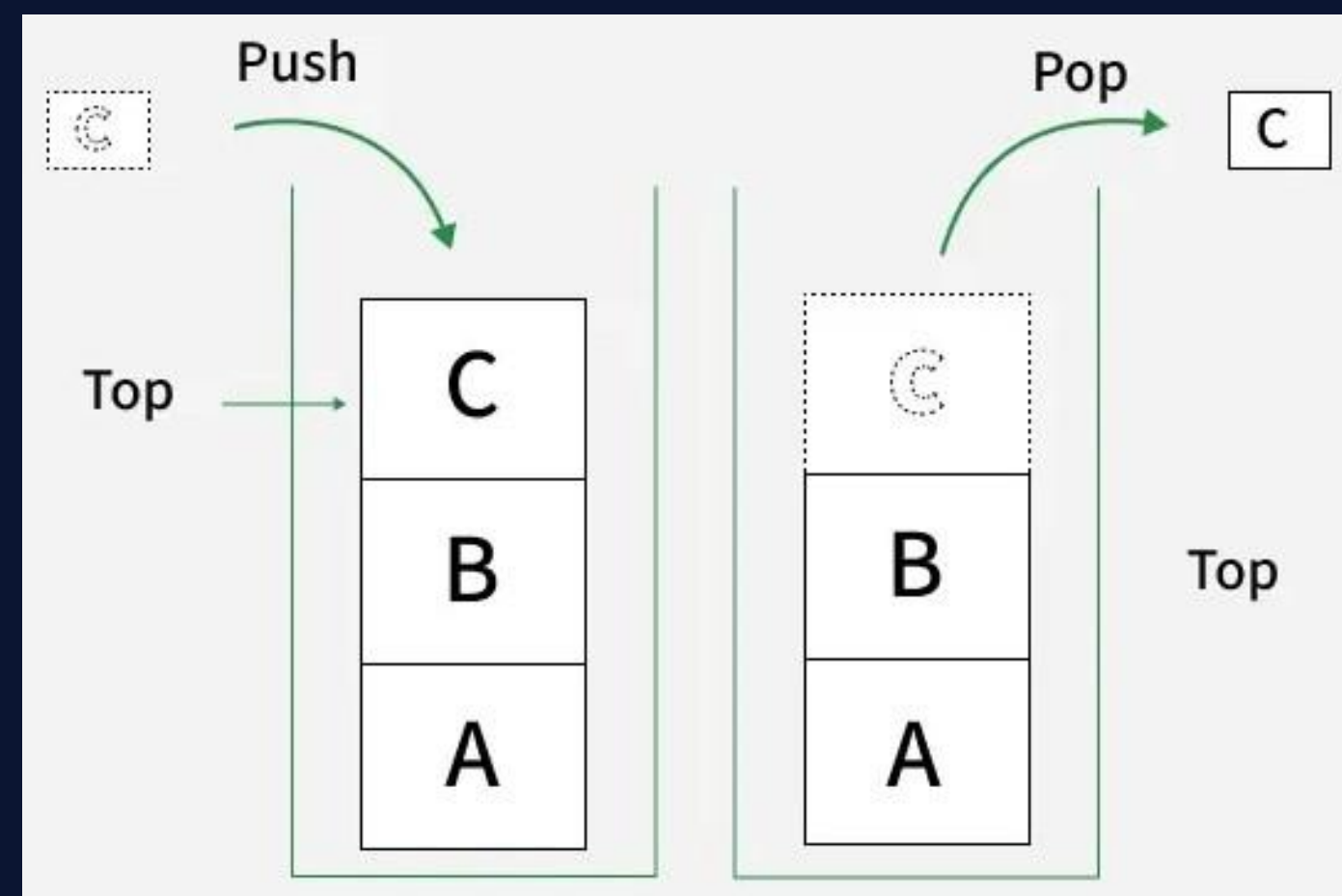
 不換

?

 [1, 2, 3, 5]

堆疊 (Stack) - 後進先出

- 概念：Last In, First Out (LIFO)
- 生活比喻：疊盤子、洋芋片桶、瀏覽器的「上一頁」
- 主要操作：
 - Push (推入)：將資料放最上面
 - Pop (彈出)：將最上面的資料拿走
- Python 實作：使用 List
 - Push `list.append(x)`
 - Pop `list.pop()` (預設拿最後一個)



圖片來源

1. 準備一個空的 List 當作堆疊

stack = []

2. 推入資料 (Push)：全部都用 append 加在尾巴

stack.append(10)

stack.append(20)

stack.append(30)

print("現在的堆疊:", stack)

結果: [10, 20, 30]

3. 彈出資料 (Pop)：用 pop() 預設就是拿「最後一個」

take = stack.pop()

print("拿走了:", take)

結果: 30 (因為它是最後放進去的)

print("剩下的:", stack) # 結果: [10, 20]

堆疊 (Stack) – 後進先出

舉例：

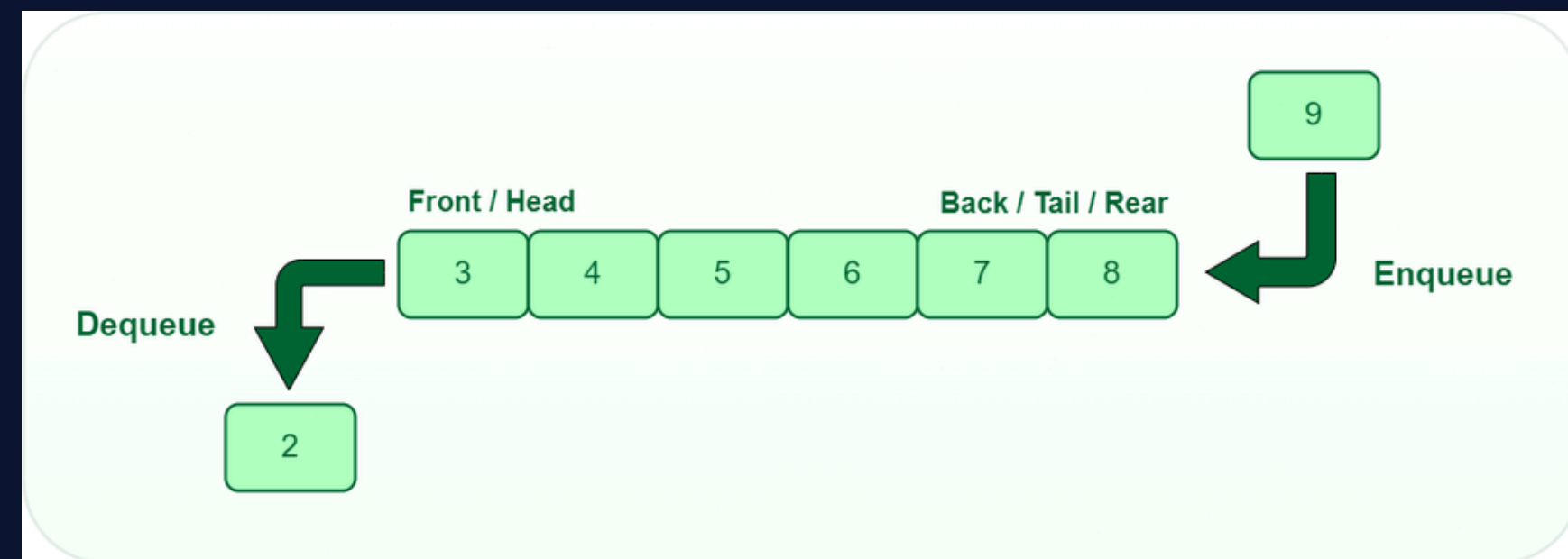
執行下列操作後，Stack 內的內容為何？(由底到頂)

```
S = []          # 建立空堆疊
S.append(1)      # [1]
S.append(2)      # [1, 2]
S.pop()          # 彈出 2 -> [1]
S.append(3)      # [1, 3]
S.append(4)      # [1, 3, 4]
S.pop()          # 彈出 4 -> [1, 3]
```

答案：[1, 3]

佇列 (Queue) - 先進先出

- 概念：First In, First Out (FIFO)
- 生活比喻：排隊買珍奶、印表機的待印文件
- 主要操作：
 - Enqueue (入列)：排在隊伍尾巴
 - Dequeue (出列)：隊伍頭的人離開
- Python 實作：使用 List (簡易版)
 - Enqueue `list.append(x)`
 - Dequeue `list.pop(0)` (拿出索引 0 的元素)



圖片來源

1. 準備一個空的 List 當作佇列

queue = []

2. 入列 (Enqueue)：一樣用 append 排在隊伍尾巴

queue.append(10)

queue.append(20)

queue.append(30)

print("現在的隊伍:", queue)

結果: [10, 20, 30]

3. 出列 (Dequeue)：關鍵在 pop(0)！拿「第 0 個」(最前面的)

take = queue.pop(0)

print("拿出:", take)

結果: 10 (因為它排第一個)

print("剩下的:", queue)

結果: [20, 30]

Stack vs Queue 比較表

特性	Stack (堆疊)	Queue (佇列)
縮寫	LIFO (後進先出)	FIFO (先進先出)
開口	單邊開口 (桶子)	雙邊開口 (水管)
加入資料	append()	append()
移除資料	pop() (拿尾端)	pop(0) (拿開頭)
應用	遞迴、復原功能(Undo)	緩衝區、排程

題目：有一個陣列 $A = [1, 2, 3, 4, 5]$

我們依序讀取 A 的數字：

- 如果是奇數 $\boxed{?}$ 放入 `Stack`
- 如果是偶數 $\boxed{?}$ 放入 `Queue`

讀取完畢後，先將 `Stack` 全部印出，再將 `Queue` 全部印出。請問輸出結果？

題目：有一個陣列 $A = [1, 2, 3, 4, 5]$

我們依序讀取 A 的數字：

- 如果是奇數

?

 放入 Stack
- 如果是偶數

?

 放入 Queue

讀取完畢後，先將 Stack 全部印出，再將 Queue 全部印出。請問輸出結果？

解析步驟：

- 讀 1 (奇)

?

 Stack: [1]
- 讀 2 (偶)

?

 Queue: [2]
- 讀 3 (奇)

?

 Stack: [1, 3]
- 讀 4 (偶)

?

 Queue: [2, 4]
- 讀 5 (奇)

?

 Stack: [1, 3, 5]

輸出階段：

Stack (LIFO) 依序彈出：5, 3, 1

Queue (FIFO) 依序出列：2, 4

答案：5 3 1 2 4